

AI USAGE DISCLOSURE FORM

OpenClaw Hackathon · AlgoTrade AI Project

1. Team Details

Team Name: AlgoTrade AI

Project / Product Name: AlgoTrade AI — Full-Stack Algorithmic Trading Simulator

Organization / Institution: RVCE

Submission Date: 07 May 2026

2. AI Usage Declaration

Did your team use any Artificial Intelligence (AI) in developing this project? **YES**

AI tools (primarily Claude) were used extensively throughout development for code generation, debugging. All AI-generated code was reviewed, tested, and modified before integration.

3. Purpose of AI Usage

☒ **Idea generation / brainstorming** : Architecture design — ML + RL + risk management + dashboard structure

☒ **Code generation or assistance** : Primary use — backtester, dashboard pages, data loaders, utils

☒ **UI / UX design** : Streamlit dark theme CSS, page layouts, Plotly chart styling

☒ **Content creation** : No AI usage

☒ **Data analysis** : Feature engineering, performance metrics, SHAP explainability

☒ **Testing / debugging** : Fixing Plotly add_vline bug, timezone stripping, cache invalidation

☐ **Other**

4. Feature Origin Classification

#	Feature Name	Origin	AI Tool / Prompt / Output / Modification
1	ML Prediction Engine (XGBoost, RF, Logistic)	Both	Tool: Claude (Anthropic). Prompt: 'Write train_model.py that trains XGBoost, Random Forest, and Logistic Regression on processed stock features and saves .pkl files.' Output: full training pipeline with 80/20 time-series split. Modification: adjusted feature columns (return_1d, MA_10, MA_50, volatility, volume_change, RSI), added TimeSeriesSplit CV, tuned XGBoost hyperparameters manually.
2	Daily Signal Page (Live BUY/HOLD/SELL)	Both	Tool: Claude (Anthropic). Prompt: 'Build a Streamlit page that fetches live Yahoo Finance data, computes features identical to training, and shows a live BUY/HOLD/SELL signal using trained .pkl models.' Output: Daily_Signal.py + live_feed.py. Modification: fixed timezone stripping bug (America/New_York tz_localize), replaced add_vline with add_shape to fix Plotly annotation crash, increased lookback to 730 days.
3	SHAP Explainability	Both	Tool: Claude (Anthropic). Prompt: 'Add SHAP feature contribution bars to the daily signal page using TreeExplainer for XGBoost/RF and LinearExplainer for Logistic Regression.' Output: SHAP integration code with fallback for missing package. Modification: added graceful fallback to raw feature values when SHAP is unavailable, fixed shap_values list indexing for binary classifiers.
4	AI Reasoning Cards (Text Explanations)	Both	Tool: Claude (Anthropic). Prompt: 'Write an explain_signal_text() function that returns plain English explanations for RSI, MACD, Bollinger Bands, MA crossovers, and model confidence.' Output: explain_signal_text() in live_feed.py returning (text, sentiment) tuples. Modification: adjusted sentiment thresholds, rewrote wording for clarity, added Bollinger %B calculation.
5	Streamlit Dashboard (8-page app)	Both	Tool: Claude (Anthropic). Prompt: 'Build a multi-page Streamlit dashboard with dark theme for ML prediction, RL agent, portfolio overview, Monte Carlo, VaR, portfolio optimizer, price indicators, and trade log pages.' Output: full page files + utils.py with shared sidebar. Modification: custom CSS theme designed independently (dark brown-orange palette), sidebar_controls() logic, engine routing (ML/RL/Both), Run Simulation cache clearing.
6	Backtester & Performance Metrics	Both	Tool: Claude (Anthropic). Prompt: 'Write backtester.py and performance_metrics.py that simulate portfolio from buy/sell signals and compute Sharpe, VaR, max drawdown, win rate, Calmar, Sortino.' Output: full backtesting engine with real P&L round-trip calculation. Modification: verified P&L calculation

#	Feature Name	Origin	AI Tool / Prompt / Output / Modification
			logic, adjusted transaction cost application, added <code>calc_pnl()</code> for accurate BUY-SELL pairing.
7	Portfolio Optimiser (Efficient Frontier)	Both	Tool: Claude (Anthropic). Prompt: 'Implement Modern Portfolio Theory with 1000 random portfolio simulations, plot efficient frontier, find Max Sharpe and Min Volatility.' Output: <code>efficient_frontier()</code> function + <code>Portfolio_Optimizer.py</code> page. Modification: fixed zero-volatility crash guard, added NaN/infinite Sharpe filter, added optimal weight percentage pill to individual stock cards.
8	Monte Carlo Simulation	Both	Tool: Claude (Anthropic). Prompt: 'Build a Monte Carlo page that simulates up to 5000 GBM price paths using real historical mu/sigma from Yahoo Finance data and shows fan chart + distribution.' Output: <code>Monte_Carlo.py</code> with <code>run_sim()</code> cached function. Modification: wired <code>mc_n</code> and <code>var_c</code> sliders so changing them instantly recalculates without pressing Run Simulation.
9	Value at Risk (VaR) Page	Both	Tool: Claude (Anthropic). Prompt: 'Build Risk VaR page with Historical VaR, Parametric VaR, CVaR, rolling 60-day VaR chart, and EWMA volatility. All should update when <code>var_c</code> slider changes.' Output: <code>Risk_VaR.py</code> . Modification: removed scipy dependency (replaced with z-score lookup table), fixed parametric VaR z-score to dynamically use <code>var_c</code> slider value.
10	RL Agent Strategy	Both	Tool: Claude (Anthropic). Prompt: 'Implement RL trading agent using momentum signals from real price data as a proxy for PPO, without running PyTorch at runtime.' Output: <code>run_rl()</code> in <code>utils.py</code> using 5-day rolling momentum. Modification: original <code>rl_agent.py</code> (DQN with PyTorch) was too slow for live dashboard use; replaced with momentum proxy that runs in milliseconds while preserving the PPO training architecture in <code>ml_strategy_rl/</code> for offline training.
11	Feature Engineering Pipeline	Self-Generated	Designed independently: identified the 6 features (<code>return_1d</code> , <code>MA_10</code> , <code>MA_50</code> , <code>volatility</code> , <code>volume_change</code> , <code>RSI</code>) based on technical analysis knowledge. Wrote the feature computation logic in <code>_compute_features()</code> ensuring exact match between training pipeline and live inference pipeline to prevent data leakage or feature mismatch.

5. Ethical & Compliance Confirmation

AI usage complies with hackathon guidelines and policies: **YES**

No proprietary or copyrighted data was misused: **I AGREE**

All data used in this project is sourced from Yahoo Finance via the public yfinance Python library. All AI-generated code was reviewed, understood, and modified by the developer. No AI output was submitted as-is without human oversight.

6. Declaration & Sign-Off

Name of Team Representative: Varshini and Chandrakala

Role: Solo Developers — ML Engineer & Full-Stack Developer

Signature: Varshini and Chandrakala

Date: 07 May 2026